

MARAN PMS — Prompt 48: Restaurant — Ajustes Operativos

Archivo principal

- `client/src/pages/restaurant.tsx` — **RestaurantPage** (componente único con todo el módulo)
- `server/routes.ts` — **endpoints restaurant**
- `server/storage.ts` — **funciones de restaurant**
- `shared/schema.ts` — **tableReservations**

Criterio general

Priorizar velocidad operativa. No romper funcionalidades existentes. Cambios incrementales.

PRIORIDAD ALTA

1. Fix: borrar items no funciona

Causa raíz: el backend `DELETE /api/restaurant/orders/:orderId/items/:itemId` devuelve `HTTP 204 No Content` (sin body). El frontend hace `res.json()` sobre esa respuesta vacía, lo que lanza una excepción y la mutación entra en error silencioso.

Fix en frontend (`deleteItemMutation`):

```
const deleteItemMutation = useMutation({
  mutationFn: async (data: { orderId: string; itemId: string }) => {
    const res = await apiRequest("DELETE", `/api/restaurant/orders/${data.orderId}`);
    // 204 no tiene body - no llamar res.json()
    if (res.status === 204 || res.status === 200) return { success: true };
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
  }
});
```

```

    toast({ title: "Item eliminado" });
  },
  onError: () => {
    toast({ title: "Error al eliminar item", variant: "destructive" });
  },
});

```

2. Buscador de ítems en comanda (búsqueda por texto)

Objetivo: el mozo puede escribir las primeras letras del plato y aparece filtrado inmediatamente, sin necesidad de navegar por categorías.

Agregar estado de búsqueda:

```
const [menuSearch, setMenuSearch] = useState("");
```

En la vista `orderView === "menu" && !pendingItem`, agregar un input de búsqueda ANTES de los botones de categoría:

```

{/* Buscador siempre visible en la comanda */}
<div className="relative">
  <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-
  <Input
    ref={menuSearchRef} // para auto-focus
    value={menuSearch}
    onChange={(e) => {
      setMenuSearch(e.target.value);
      if (e.target.value) setSelectedCategory(null); // búsqueda sobrescribe cat
    }}
    placeholder="Buscar plato o código..."
    className="pl-9"
    data-testid="input-menu-search"
  />
  {menuSearch && (
    <button
      className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground
      onClick={() => setMenuSearch("")}
    >
      <X className="h-4 w-4" />
    </button>
  )}
</div>

```

Lógica de filtrado:

```
// Items visibles según búsqueda o categoría seleccionada
const visibleItems = menuSearch.trim()
  ? menuItems.filter(item =>
    item.isAvailable !== "false" && (
      item.name.toLowerCase().includes(menuSearch.toLowerCase()) ||
      (item.description || "").toLowerCase().includes(menuSearch.toLowerCase())
    )
  )
  : selectedCategory
  ? menuItems.filter(item => item.categoryId === selectedCategory && item.isAva
  : [];

// Cuando hay búsqueda, mostrar con su categoría como subtítulo:
{menuSearch.trim() && visibleItems.length > 0 && (
  <div className="grid gap-2 sm:grid-cols-2">
    {visibleItems.map(item => {
      const cat = menuCategories.find(c => c.id === item.categoryId);
      return (
        <button key={item.id} className="p-3 border rounded-md text-left hover-el
          onClick={() => { setPendingItem(item); setMenuSearch(""); /* resto igua
            <div className="font-medium">{item.name}</div>
            <div className="text-xs text-muted-foreground">{cat?.name}</div>
            <div className="text-muted-foreground text-sm">${parseFloat(item.price)
          </button>
        );
      })}
    </div>
  )}

{menuSearch.trim() && visibleItems.length === 0 && (
  <p className="text-center text-muted-foreground py-4 text-sm">Sin resultados pa
  )}
```

Limpiar búsqueda al confirmar o cancelar item:

```
// En handleConfirmItem y handleCancelItem:
setMenuSearch("");
```

Auto-focus: crear `const menuSearchRef = useRef<HTMLInputElement>(null)` y cuando se abre `isOrderDialogOpen`, hacer `setTimeout(() => menuSearchRef.current?.focus(), 100)`.

3. Guardado de datos en apertura de mesa (evitar pérdida accidental)

Problema: si el usuario hace clic fuera del dialog de "Nuevo Pedido", el estado `newWaiterName` y `newCovers` se resetean al cerrar.

Fix: cambiar el comportamiento del dialog para que NO cierre al hacer clic fuera:

```
// En el Dialog de apertura de mesa:
<Dialog
  open={isNewOrderDialogOpen}
  onOpenChange={(open) => {
    // Solo cerrar si el usuario presiona Cancelar explícitamente
    // NO cerrar al hacer clic fuera
    if (open) setIsNewOrderDialogOpen(true);
    // No setear false aquí - solo el botón Cancelar lo hace
  }}
>
  <DialogContent
    onPointerDownOutside={(e) => e.preventDefault()} // bloquea cierre por clic e
    onEscapeKeyDown={(e) => e.preventDefault()}      // bloquea cierre por ESC
  >
```

Aplicar el mismo patrón al `isDirectOrderDialogOpen`.

El botón "Cancelar" sí debe cerrar y limpiar:

```
<Button variant="outline" onClick={() => {
  setIsNewOrderDialogOpen(false);
  setNewWaiterName("");
  setNewCovers(2);
}}>
  Cancelar
</Button>
```

4. Listado completo de reservas con selector de fecha

Problema actual: `isDailyReservationsOpen` muestra solo las reservas del día (`todayReservations`).

Fix: agregar selector de modo "Por día / Todas" con buscador:

```
const [reservationViewMode, setReservationViewMode] = useState<"day" | "all">("da
const [reservationDateFilter, setReservationDateFilter] = useState(
  new Date().toLocaleDateString("en-CA", { timeZone: "America/Argentina/Buenos_Ai
```

```

);
const [reservationSearchText, setReservationSearchText] = useState("");

// Filtrado:
const filteredReservations = reservations.filter(r => {
  if (r.status === "cancelled") return false;
  if (reservationViewMode === "day" && r.reservationDate !== reservationDateFilter)
  if (reservationSearchText) {
    const q = reservationSearchText.toLowerCase();
    return (
      r.guestName.toLowerCase().includes(q) ||
      (r.guestPhone || "").includes(q) ||
      (r.guestEmail || "").toLowerCase().includes(q) ||
      r.reservationDate.includes(q)
    );
  }
  return true;
});

```

En el dialog de reservas diarias, agregar arriba del listado:

```

{/* Selector de modo */}
<div className="flex gap-2 items-center">
  <Button
    size="sm"
    variant={reservationViewMode === "day" ? "default" : "outline"}
    onClick={() => setReservationViewMode("day")}
  >
    Por día
  </Button>
  <Button
    size="sm"
    variant={reservationViewMode === "all" ? "default" : "outline"}
    onClick={() => setReservationViewMode("all")}
  >
    Todas
  </Button>
  {reservationViewMode === "day" && (
    <Input
      type="date"
      value={reservationDateFilter}
      onChange={(e) => setReservationDateFilter(e.target.value)}
      className="h-8 w-40"
    />
  )}
</div>

```

```

{/* Buscador */}
<Input
  placeholder="Buscar por nombre, teléfono o email..."
  value={reservationSearchText}
  onChange={(e) => setReservationSearchText(e.target.value)}
  className="mt-2"
/>

```

En la lista mostrar también la fecha cuando `reservationViewMode === "all"` :

```

// En cada fila de reserva, agregar fecha si es vista "todas":
{reservationViewMode === "all" && (
  <span className="text-xs text-muted-foreground">{formatDateAR(r.reservationDate
)}

```

PRIORIDAD MEDIA

5. Aclarar “Crear pedido” y “Número de orden”

Fix UX — tooltips informativos:

En el botón “Crear Pedido” del dialog de apertura de mesa, agregar una nota explicativa debajo del título:

```

<DialogHeader>
  <DialogTitle>Nuevo Pedido – Mesa {selectedTable?.tableNumber}</DialogTitle>
  <DialogDescription>
    Al crear el pedido se abre la mesa y se habilita la carga de comandas.
  </DialogDescription>
</DialogHeader>

```

Donde se muestra el `orderNumber` **en la UI (encabezado de orden, folio, etc.), agregar tooltip:**

```

<span className="font-mono text-xs text-muted-foreground" title="Número interno d
  #{order.orderNumber}
</span>

```

El `orderLabel` (nombre de la orden) es lo que el equipo asigna libremente. El `orderNumber` es generado automáticamente por el sistema. Dejar esto claro en la

descripción del dialog.

6. Curso automático desde categoría (no pedir si ya está definido)

Problema: al agregar un item, siempre se muestra la selección de curso (Entrada/Principal/Postre) aunque la categoría ya lo define.

Fix: en el schema de `menuCategories`, cada categoría puede tener un `defaultCourse` (1=Entrada, 2=Principal, 3=Postre, null=sin curso). Por ahora, inferirlo del nombre de la categoría:

```
function inferCourseFromCategory(categoryName: string): number | null {
  const name = categoryName.toLowerCase();
  if (name.includes("entrada") || name.includes("aperitivo")) return 1;
  if (name.includes("principal") || name.includes("segundo") || name.includes("pl
  if (name.includes("postre") || name.includes("dulce")) return 3;
  if (name.includes("bebida") || name.includes("cerveza") || name.includes("vino"
  return null; // sin inferencia → mostrar selector
}
```

Al seleccionar un item (`setPendingItem`), inicializar `itemCourse` con el curso inferido:

```
const inferredCourse = inferCourseFromCategory(cat?.name || "");
setItemCourse(inferredCourse ?? 1);
```

Si `inferredCourse !== null`, ocultar el selector de curso en el `pendingItem`:

```
{inferCourseFromCategory(menuCategories.find(c => c.id === pendingItem.categoryId
  <div className="space-y-2">
    <Label>Curso</Label>
    {/* selector de curso */}
  </div>
)}
```

7. Observaciones al final del flujo de comanda

Problema: el campo de observaciones aparece antes del botón de confirmación, interrumpiendo el flujo para items que no necesitan nota.

Fix UX: colapsar observaciones por defecto, expandir con un botón:

```

const [showItemNotes, setShowItemNotes] = useState(false);

// En el pendingItem view, reemplazar el textarea por:
<button
  className="text-sm text-muted-foreground underline-offset-2 hover:underline"
  onClick={() => setShowItemNotes(!showItemNotes)}
>
  {showItemNotes ? "Ocultar observaciones" : "+ Agregar observación"}
</button>
{showItemNotes && (
  <Textarea
    value={itemNotes}
    onChange={(e) => setItemNotes(e.target.value)}
    placeholder="Ej: sin sal, término medio, sin gluten..."
    rows={2}
    autoFocus
    data-testid="input-item-notes"
  />
)}

```

Resetear `showItemNotes` **al confirmar o cancelar item:**

```

// En handleConfirmItem y handleCancelItem:
setShowItemNotes(false);
setItemNotes("");

```

8. Factura C — ocultar o mover a configuración admin

Objetivo: simplificar las opciones del cierre de mesa para el personal de salón.

Fix: sacar Factura C de `receiptTypeLabels` por defecto y agregarla solo si una config de sistema lo habilita. Como solución inmediata, mover Factura C al final del selector y agregar una nota:

```

// En receiptTypeLabels, reordenar:
const receiptTypeLabels: Record<string, string> = {
  ticket: "Ticket",
  factura_a: "Factura A",
  factura_b: "Factura B",
  voucher: "Voucher (No Fiscal)",
  factura_c: "Factura C", // ← mover al final
};

```


Alternativa preferida (más limpia): agregar una constante de control:

```
const SHOW_FACTURA_C = false; // cambiar a true en configuración admin si se nece

// En el Select de comprobante:
{Object.entries(receiptTypeLabels)
  .filter(([value]) => value !== "factura_c" || SHOW_FACTURA_C)
  .map(([value, label]) => (
    <SelectItem key={value} value={value}>{label}</SelectItem>
  ))
}
```

9. Anticipos en reservas de restaurant

Objetivo: registrar y actualizar anticipos (señas) en reservas de mesa.

Migración DB:

```
ALTER TABLE table_reservations
  ADD COLUMN advance_amount DECIMAL(10,2) DEFAULT 0,
  ADD COLUMN advance_method TEXT,
  ADD COLUMN advance_date DATE,
  ADD COLUMN advance_notes TEXT;
```

Schema (shared/schema.ts):

```
export const tableReservations = pgTable("table_reservations", {
  // ... campos existentes ...
  advanceAmount: decimal("advance_amount", { precision: 10, scale: 2 }).default("
  advanceMethod: text("advance_method"),
  advanceDate: date("advance_date"),
  advanceNotes: text("advance_notes"),
});
```

En el formulario de edición de reserva de restaurant, agregar sección de anticipo:

```
<div className="border-t pt-4 space-y-3">
  <Label className="text-sm font-medium">Anticipo / Señal</Label>
  <div className="grid grid-cols-2 gap-3">
    <div>
      <Label className="text-xs">Monto</Label>
      <Input
        type="number"
```

```

        step="0.01"
        value={editingReservation.advanceAmount || ""}
        onChange={(e) => setEditingReservation({ ...editingReservation, advanceAm
        placeholder="0.00"
    />
</div>
<div>
    <Label className="text-xs">Método</Label>
    <Select
        value={editingReservation.advanceMethod || ""}
        onChange={(v) => setEditingReservation({ ...editingReservation, adva
    >
        <SelectTrigger><SelectValue placeholder="Forma de pago" /></SelectTrigger>
        <SelectContent>
            <SelectItem value="efectivo">Efectivo</SelectItem>
            <SelectItem value="transferencia">Transferencia</SelectItem>
            <SelectItem value="tarjeta_debito">Tarjeta Débito</SelectItem>
            <SelectItem value="tarjeta_credito">Tarjeta Crédito</SelectItem>
            <SelectItem value="mercadopago">MercadoPago</SelectItem>
        </SelectContent>
    </Select>
</div>
</div>
<div>
    <Label className="text-xs">Notas del anticipo</Label>
    <Input
        value={editingReservation.advanceNotes || ""}
        onChange={(e) => setEditingReservation({ ...editingReservation, advanceNote
        placeholder="Referencia, comprobante, observación..."
    />
</div>
{ /* Mostrar estado del anticipo */ }
{ (editingReservation.advanceAmount || 0) > 0 && (
    <div className="text-xs text-muted-foreground bg-muted/50 rounded p-2">
        Anticipo registrado: ${editingReservation.advanceAmount?.toLocaleString()}
        {editingReservation.advanceMethod && ` - ${editingReservation.advanceMethod
    </div>
    ) }
</div>

```

En el listado de reservas, mostrar badge "Seña" cuando `advanceAmount > 0` :

```

{ (r.advanceAmount || 0) > 0 && (
    <Badge variant="outline" className="text-xs text-green-700 border-green-400">
        Seña ${r.advanceAmount?.toLocaleString()}
    </Badge>

```

```
}}
```

10. Datos de facturación al cierre de mesa

Objetivo: cuando el cliente pide comprobante (Factura A o B), poder ingresar CUIT/razón social.

Agregar estado:

```
const [closeBillingName, setCloseBillingName] = useState("");
const [closeBillingCuit, setCloseBillingCuit] = useState("");
```

En el dialog de cierre, mostrar campos cuando el comprobante es Factura A o B:

```
((closeReceiptType === "factura_a" || closeReceiptType === "factura_b") && (
  <div className="space-y-3 p-3 border rounded-md bg-muted/30">
    <p className="text-sm font-medium">Datos de facturación</p>
    <div>
      <Label className="text-xs">Razón social / Nombre</Label>
      <Input
        value={closeBillingName}
        onChange={(e) => setCloseBillingName(e.target.value)}
        placeholder="Ej: Juan García / Empresa SA"
        data-testid="input-billing-name"
      />
    </div>
    <div>
      <Label className="text-xs">CUIT / DNI</Label>
      <Input
        value={closeBillingCuit}
        onChange={(e) => setCloseBillingCuit(e.target.value)}
        placeholder="20-12345678-9"
        data-testid="input-billing-cuit"
      />
    </div>
  </div>
))
```

Pasar estos datos al `closeOrderMutation`:

```
closeOrderMutation.mutate({
  orderId: currentOrder.id,
  receiptType: closeReceiptType,
```

```
paymentMethod: closePaymentMethod,  
discount: ...,  
discountType: ...,  
roomReservationId: ...,  
billingName: closeBillingName || undefined,  
billingCuit: closeBillingCuit || undefined,  
});
```

El backend puede registrar estos datos en el log o en el campo `notes` de la orden por ahora.

Limpiar al cerrar el dialog:

```
setCloseBillingName("");  
setCloseBillingCuit("");
```

NOTAS TÉCNICAS

- El fix #1 (borrar items) es el más crítico — solo requiere cambiar `res.json()` por el manejo de 204.
- El fix #2 (buscador) no reemplaza las categorías — ambas formas de navegar coexisten.
- Para el fix #3 (guardar apertura de mesa), `onPointerDownOutside` y `onEscapeKeyDown` son props del componente `DialogContent` de `shadcn/ui` — ya disponibles.
- Los anticipos (#9) requieren migración DB. Usar `DEFAULT 0` y `DEFAULT NULL` para no afectar reservas existentes.
- Los datos de facturación (#10) no requieren DB por ahora — se pueden pasar como metadata al endpoint de cierre y logear. La integración con ARCA/AFIP para factura electrónica es una etapa posterior.
- No agregar `requireAuth` a endpoints ya existentes sin verificar que no rompan el flujo actual — los endpoints del restaurant pueden no tener auth en algunos casos.